Phase 06 — Docker and Containers (Complete Notes)

> *"It works on my machine" — Docker makes that sentence mean something.*

---

1. Why containers exist

The eternal problem: your app runs on your laptop (Java 21, PostgreSQL 16, certain env vars) and breaks on the server (Java 17, different libs, missing config). Environments drift; deployments become archaeology.

Old fix — **virtual machines**: ship a whole OS per app. Isolated, but heavy: GBs each, minutes to boot, an OS kernel each.

Docker's fix — **containers**: package the app + all its dependencies + config into an **image**; run it anywhere with the *same* behavior. Containers share the host's Linux kernel and isolate at the process level (namespaces + cgroups) — megabytes, milliseconds to start.

```
VMs:                              CONTAINERS:
[app][app]                        [app][app][app]
[OS ][OS ] ← full OS each         [ Docker Engine ]
[hypervisor]                      [ one host OS kernel ]
[hardware]                        [ hardware ]
```

Analogy: VMs = each merchant ships goods in their own whole ship. Containers = standardized shipping containers: any goods inside, same shape outside — every crane, truck, and port (any server, any cloud) handles them identically.

2. Docker architecture & core objects

- **Docker Engine (daemon)** — background service managing everything.
- **Image** — read-only template/blueprint: filesystem + app + deps (class ↔ recipe).
- **Container** — a *running instance* of an image (object ↔ cooked dish). Many containers from one image.
- **Registry** — image store: Docker Hub (public), ECR/GHCR (private). `docker pull/push`.
- Images are built in **layers**; layers are cached and shared → fast rebuilds, small transfers.

```
docker run hello-world                      # pull + create + start
docker run -d -p 8080:80 --name web nginx   # -d background, -p host:container port map
docker ps            # running containers    docker ps -a   # incl. stopped
docker logs -f web   # container's stdout — THE debugging tool
docker exec -it web bash                    # shell INSIDE the container
docker stop web && docker rm web
docker images; docker rmi nginx
```

`-p 8080:80` = host port 8080 → container port 80. Containers have isolated network namespaces; without `-p`, nothing outside reaches them.

3. Dockerfile — image as code

`Dockerfile` for a Spring Boot app (multi-stage: build with Maven, ship only the JRE + jar):

```
# ---- build stage ----
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline          # layer cached unless pom.xml changes ★
COPY src ./src
RUN mvn package -DskipTests

# ---- run stage (small, no maven/src inside) ----
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
USER 1000                              # don't run as root (Phase 4 principle!)
ENTRYPOINT ["java", "-jar", "app.jar"]
```

```
docker build -t shop-api:1.0 .
docker run -d -p 8080:8080 -e SPRING_PROFILES_ACTIVE=prod shop-api:1.0
```

Layer-caching rule: **order instructions from least → most frequently changing** (deps before source), so code edits rebuild only the last layers.

4. Networks

Containers on the same user-defined network reach each other **by container name** (Docker runs an internal DNS — Phase 2 in miniature!):

```
docker network create appnet
docker run -d --network appnet --name db -e POSTGRES_PASSWORD=secret postgres:16
docker run -d --network appnet --name api -p 8080:8080 shop-api:1.0
# inside 'api', the DB hostname is literally:  db:5432
# spring.datasource.url=jdbc:postgresql://db:5432/shop      ← name, not IP, not localhost!
```

★ Classic bug: `localhost` inside a container = **that container itself**, not your machine, not another container. DB connection refused? You wrote `localhost:5432` instead of `db:5432`.

5. Volumes — data that survives

A container's filesystem dies with the container. Databases need **volumes** — host-managed storage mounted inside:

```
docker volume create pgdata
docker run -d --name db -v pgdata:/var/lib/postgresql/data postgres:16   # named volume
docker run -v $(pwd)/nginx.conf:/etc/nginx/nginx.conf:ro nginx           # bind mount (config)
```

Rule: **stateless containers, state in volumes** (or better: managed DBs — Phase 8).

6. Docker Compose — the whole stack in one file

Running 4 containers with flags is unmanageable. Compose declares the stack in YAML:

`docker-compose.yml` — our course architecture:

```
services:
  nginx:
    image: nginx:1.27
    ports: ["80:80", "443:443"]                    # ONLY nginx is exposed
    volumes:
      - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on: [api]

  api:
    build: .                                       # uses our Dockerfile
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/shop
      SPRING_DATA_REDIS_HOST: redis
    depends_on:
      db: { condition: service_healthy }
    # note: NO ports: — reachable only inside the network, via nginx ★

  db:
    image: postgres:16
    environment:
      POSTGRES_DB: shop
      POSTGRES_PASSWORD: ${DB_PASSWORD}            # from .env file — never hardcode
    volumes: [pgdata:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      retries: 10

  redis:
    image: redis:7

volumes:
  pgdata:
```

nginx.conf inside the compose network proxies by service name:

```
location / { proxy_pass http://api:8080; }
```

```
docker compose up -d          # build + network + start everything
docker compose ps             # status
docker compose logs -f api    # logs per service
docker compose down           # stop & remove (data survives in volumes)
docker compose down -v        # ...including volumes (careful!)
```

This file **is** the architecture diagram from the roadmap, executable:

```
NGINX container → Spring Boot container → PostgreSQL container
                                        → Redis container
```

7. Real-world production use

- Same image runs in dev → staging → prod; only env vars differ (12-factor principle).
- CI builds the image once, tags it (`shop-api:git-sha`), pushes to a registry; servers just `pull` and `run` — deployment becomes *replacing a container*, seconds, reversible.
- Compose is fine for one-server production; many servers need orchestration → Kubernetes (Phase 11).

8. Common mistakes

- `localhost` instead of service names between containers (the #1 error).
- Running DBs without volumes → `down` deletes all data.
- Secrets hardcoded in Dockerfile/compose (bake into image = leaked forever in layers). Use env/.env/secret stores.
- Giant images: full JDK+Maven shipped to prod instead of multi-stage JRE-only; no `.dockerignore` (copies `.git` , `target/` into build context).
- `:latest` tag in production — irreproducible deploys.
- Exposing the DB port to the host "for debugging" and leaving it (bypasses your whole security model; on a public server that's a breach).
- Running as root inside containers.

9. Best practices

- Multi-stage builds; pin versions (`postgres:16` , not `latest`); one process per container.
- Healthchecks + `depends_on: condition: service_healthy` for startup ordering.
- Only the edge (nginx) publishes ports; everything else is internal.
- `.dockerignore` mirroring `.gitignore` ; `docker system prune` occasionally.

10. Interview questions

1. Container vs VM — what exactly is shared, what is isolated?
2. Image vs container?
3. How do Docker layers work and how do you order a Dockerfile for cache efficiency?
4. Why multi-stage builds for Java apps?
5. Two containers must talk — how? Why does `localhost` fail?
6. Where does database data live and what happens on `docker compose down` vs `down -v` ?
7. How would you ship a Spring Boot app from laptop to server with Docker (full flow incl. registry)?
8. Why not run as root in a container?

11. LAB

```
sudo apt install docker.io docker-compose-v2    # or Docker Desktop
sudo usermod -aG docker $USER && newgrp docker  # docker without sudo

# 1. warm-up
docker run -d -p 8080:80 --name w nginx; curl localhost:8080; docker logs w; docker exec -it w bash

# 2. build the full stack from §6 with a real Spring Boot app
#    (any CRUD app with JPA + Redis, or start from start.spring.io: web, data-jpa, redis, postgres
driver)
docker compose up -d --build
docker compose ps                     # wait for healthy
curl localhost/api/...                # through NGINX → api → db!

# 3. prove the concepts
docker compose stop api; curl -i localhost/api/...      # 502 from NGINX! (Phase 5)
docker compose start api
docker compose down && docker compose up -d             # data still there? (volumes)
docker compose exec db psql -U postgres shop            # poke around inside the DB container
```

12. ASSIGNMENT 06 (submit to Claude)

1. Complete lab step 2 with a real Spring Boot app. Paste your Dockerfile, compose file, and the output of `docker compose ps` (all healthy).
2. Explain line by line what happens on `docker compose up -d` the very first time (build, network, DNS, order).
3. Your teammate's container can't reach PostgreSQL: `Connection refused to localhost:5432`. The compose file has services `app` and `postgres`. Diagnose and give the exact fix.
4. Your image is 1.4 GB. List every technique you know to shrink it, most impactful first.
5. From memory: draw the network diagram of the compose stack, marking which ports are published to the host and which are internal-only, and why that boundary matters for security.